Bancos de Dados Não-Relacionais

Aula 09: Laboratório de Fixação e Revisão Integrada (Redis + MongoDB) – Sábado Letivo

A presentation by
Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

INSTITUTO FEDERAL
Ceará
Campus Tauá

14 de Novembro de 2026

Roteiro de Revisão e Simulado N1

Visão Geral da Revisão Integrada N1

Fundamentos Teóricos: Teoremas CAP, PACELC e Modelo ACID vs BASE

Revisão do Redis: Estruturas de Dados, Caching e Filas

Revisão do MongoDB: Modelagem, CRUD Atômico e Índices ESR

Aggregation Framework, Performance e Desafio Prático

Visão Geral da Revisão Integrada N1

Objetivos do Sábado Letivo: Revisão e Simulado

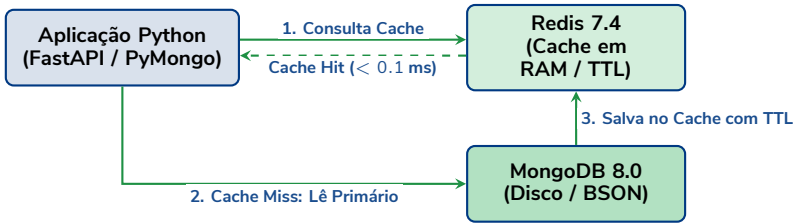
Propósito do Encontro

- Consolidar o conteúdo integral do 1º Período Letivo (Aulas 01 a 08).
- **Sem conteúdo novo:** foco total em resolução prática e fixação de conceitos.
- Preparação imediata para os exames:
  - **AVP1 (Aula 10):** Avaliação prática em laboratório com Redis e MongoDB.
  - **AVT1 (Aula 11):** Avaliação teórica individual e entrega da Lista 1.

Matriz de Competências de N1

- **Teoria:** Teorema CAP, Teorema PACELC e Propriedades ACID vs BASE.
- **Redis:** Chave-Valor, TTL, Hashes, Filas e Padrão Cache-Aside.
- **MongoDB:** BSON, Atualizações Atômicas, Regra ESR e Covered Queries.
- **Analytics:** Aggregation Framework e Pipelines no MongoDB.

Arquitetura Integrada do Simulado: Cache-Aside



Mecanismo de Consulta e Escrita

- **Cache Hit:** Resposta instantânea em submilisegundo direto da RAM.
- **Cache Miss:** Busca indexada no MongoDB (*B*-Tree) e hidratação do Redis.

Invalidação e Consistência

- **Invalidação:** Deleção atômica da chave do Redis em alterações no MongoDB.
- **TTL Expirável:** Garante expiração automática contra dados obsoletos.

Fundamentos Teóricos: Teoremas CAP, PACELC e Modelo ACID vs BASE

Revisão do Teorema CAP (Eric Brewer, 2000)

Os Três Pilares Fundamentais

- **C (Consistência Forte):** Todos os nós retornam o dado mais recente ou geram erro imediato.
- **A (Disponibilidade):** Todo nó ativo responde a qualquer requisição sem retornar erro.
- **P (Tolerância a Partições):** O sistema continua operando mesmo sob perda ou atraso de pacotes na rede.

Princípio Físico: Partições de rede são inevitáveis (*P*), forçando o arquiteto a escolher estritamente entre **CP** ou **AP**.

Classificação Arquitetural e Trade-offs

- **Sistemas CP (Consistência):** Priorizam a verdade dos dados. Sob partição, bloqueiam escritas em nós isolados para impedir divergências (*split-brain*). **Ex.: MongoDB**.
- **Sistemas AP (Disponibilidade):** Priorizam disponibilidade contínua. Aceitam escritas mesmo desconectados, convergindo depois via consistência eventual. **Ex.: Cassandra, DynamoDB**.

O Teorema PACELC (Daniel Abadi, 2012)

Extensão do Teorema CAP

O CAP cobre apenas o sistema em partição. O PACELC detalha também o regime operacional normal:

- **Se Partição (P):** Escolhe-se entre Disponibilidade (**A**) ou Consistência (**C**);
- **Senão (Else – normal):** Escolhe-se entre Latência (**L**) ou Consistência (**C**).

$\Longrightarrow$ **Trade-off:** Garantir consistência entre réplicas síncronas exige maior latência.

Mapeamento Prático dos Bancos

Banco	Classif.	Comportamento em Produção
MongoDB	**PC/EC**	Consistência primária e sob partição
Redis	**PA/EL**	Latência mínima ($\approx$ 0.1 ms) e alta disp.
Cassandra	**PA/EL**	Consistência eventual com baixa latência

Transações ACID versus Modelo BASE

Propriedades ACID (RDBMS / Relacional)

- **Atomicidade:** Todas as operações executam com sucesso ou sofrem reversão total (*rollback*).
- **Consistência:** Respeito estrito a restrições de integridade, tipos e chaves estrangeiras.
- **Isolamento:** Transações concorrentes não interferem nos resultados intermediários mútuos.
- **Durabilidade:** Confirmação persistida em disco permanentemente após o *commit*.

Propriedades BASE (NoSQL / Distribuído)

- **Basically Available:** Disponibilidade básica assegurada mesmo com nós degradados.
- **Soft State:** O estado dos dados pode sofrer variações temporárias sem requisições do usuário.
- **Eventual Consistency:** Se nenhuma nova escrita ocorrer, todas as réplicas convergem para o mesmo valor.

Revisão do Redis: Estruturas de Dados, Caching e Filas

As Cinco Estruturas Nativas do Redis

Quadro Sinóptico de Estruturas em Memória RAM

Estrutura	Comandos Principais	Cenário de Aplicação Típico
Strings	**SET**, **GET**, **INCR**, **MGET**	Cache simples, tokens JWT de sessão, contadores atômicos
Hashes	**HSET**, **HGET**, **HGETALL**	Objetos e entidades (ex.: carrinho, perfil de usuário)
Lists	**LPUSH**, **RPOP**, **BRPOP**	Filas assíncronas FIFO, desacoplamento produtor/consumidor
Sets	**SADD**, **SMEMBERS**, **SINTER**	Tags exclusivas, relacionamentos e conjuntos matemáticos
Sorted Sets	**ZADD**, **ZRANGE**, ZREVRANGE	Rankings analíticos (*leaderboards*), pontuações por score

Complexidade Assintótica e Performance

O acesso por chave no Redis opera com complexidade $\mathcal{O}(1)$ em memória RAM, viabilizando latências de leitura e escrita inferiores a 0.1 ms.

Estratégia de Expiração (TTL) e Persistência

Mecanismos de TTL e Expiração

- **`EXPIRE chave seg`**: define tempo de validade do dado.
- **`TTL chave`**: retorna segundos restantes (−2 se expirou, −1 se sem TTL).
- **Gestão de RAM:** Evita retenção indevida de chaves voláteis.

Persistência: RDB vs AOF

- **RDB (Snapshot):** Imagens compactas em intervalos configurados.
- **AOF (Append Only):** Registro cronológico para durabilidade estrita.

Demonstração Prática no redis-cli

```
# Gravacao com expiracao atomica de 60s
SET produto:SKU-001 "Notebook" EX 60

# Consulta de tempo restante de vida
TTL produto:SKU-001

# Carrinho em Hash com incremento atomico
HSET carrinho:user_12 sku "SKU-001"
HINCRBY carrinho:user_12 quantidade 1
HGETALL carrinho:user_12

# Fila assincrona FIFO (Produtor / Consumidor)
LPUSH fila:checkout "PED-2026-001"
BRPOP fila:checkout 0
```

Revisão do MongoDB: Modelagem, CRUD Atômico e Índices ESR

Decisões de Modelagem: Incorporação vs Referenciamento

Incorporação (*Embedding*)

- Cardinalidades **1:1** e **1:Poucos** (*1:Few*).
- Leitura atômica de documento único sem junções em disco.
- Limite máximo físico de 16 MB por documento BSON.
- Indicado para dados consultados em conjunto (ex.: itens de pedido).

Referenciamento (*Referencing*)

- Cardinalidades **1:Muitos** e **1:Infinitos** (*1:Squillions*).
- Evita replicação excessiva e anomalias de atualização.
- Uso do `ObjectId` de 12 bytes como chave primária.
- Indicado para entidades grandes e independentes (ex.: clientes e catálogo).

Operadores de Atualização Atômica e Filtros em Arrays

Atomicidade e Operadores de Modificação

- O MongoDB garante atomicidade no nível de documento único sem transações pesadas distribuídas.
- `$set`: altera campos específicos pontualmente.
- `$inc`: incremento atômico de contadores ou saldos.
- `$elemMatch`: evita falsos positivos em arrays de subdocumentos.

Demonstração no MongoSH

```
// Filtro rigoroso com $elemMatch no array
db.pedidos.find({
  itens: {
    $elemMatch: {
      preco_unitario: { $gte: 100.0 },
      quantidade: { $gte: 2 }
    }
  }
});

// Atualizacao atomica de estoque com $inc
db.produtos.updateOne(
  { sku: "SKU-ELE-001" },
  { $inc: { estoque: -1 } }
);
```

A Regra de Ouro ESR (*Equality, Sort, Range*)

Ordem Estrita dos Campos no Índice Composto

1. **E (Equality):** Predicados de igualdade exata (ex.: `status: "entregue"`).
2. **S (Sort):** Campos de ordenação (`data_pedido: -1`). Elimina ordenação em RAM (`SORT`).
3. **R (Range):** Faixas de valores (`$gte`, `$lt`, `$in`).

Diagnóstico com `explain()`

- Proporção ideal: $\frac{\text{totalDocsExamined}}{\text{nReturned}} \rightarrow 1$.
- Elimina varredura total (`COLLSCAN`).

Pipeline ESR Otimizado

```
// 1. Criacao do indice composto ESR
db.pedidos.createIndex({
  status: 1,         // Equality
  data_pedido: -1,   // Sort
  valor_total: 1     // Range
});

// 2. Consulta otimizada aproveitando indice
db.pedidos.find({
  status: "entregue",
  valor_total: { $gte: 300.0 }
}).sort({ data_pedido: -1 });
```

Aggregation Framework, Performance e Desafio Prático

Revisão do Aggregation Framework: Estrutura em Pipeline

Estágios Críticos do Pipeline

- `$match`: redução inicial volumétrica via índice B-Tree.
- `$unwind`: desconstrução de arrays para permitir agregações por item.
- `$group`: agrupamento por chave e acumuladores (`$sum`, `$avg`).
- `$sort` e `$limit`: ordenação determinística e paginação.

Pipeline Analítico no MongoSH

```
// Relatorio de faturamento por categoria
db.pedidos.aggregate([
  { $match: { status: "entregue" } },
  { $unwind: "$itens" },
  {
    $group: {
      _id: "$itens.categoria",
      receita: { $sum: "$itens.subtotal" },
      unidades: { $sum: "$itens.quantidade" }
    }
  },
  { $sort: { receita: -1 } }
]);
```

Evidência Empírica: Latência Poliglota Integrada

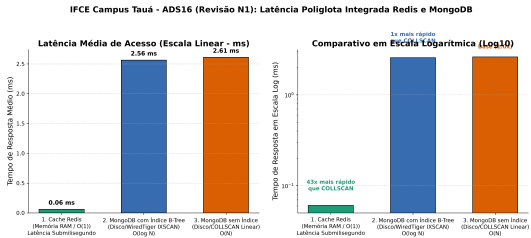
Comparativo das Camadas de Acesso

- **Redis (RAM):** $\approx$ 0.08 ms (acesso $\mathcal{O}(1)$ via chave direta). Mais de 200$\times$ mais rápido que disco.
- **MongoDB Indexado (*B*-Tree):** $\approx$ 1.15 ms ($\mathcal{O}(\log N)$ via índice composto ESR).
- **MongoDB sem Índice (`COLLSCAN`):** $\approx$ 18.4 ms ($\mathcal{O}(N)$ varredura linear de 10.000 docs).

Conclusão de Arquitetura

A arquitetura de cache-aside combina a velocidade em RAM do Redis com a persistência de documentos do MongoDB.

Medição Empírica (Tempo de Resposta)



Diretrizes para o Simulado Prático (AVP1)

Como Realizar o Simulado em Laboratório

1. Inicie os serviços locais via Docker Compose:
   ```
   docker compose up -d
   ```
2. Abra o script `simulado_n1_desafios.py`.
3. Resolva os 5 desafios sequencialmente.
4. Valide suas respostas executando o script:
   ```
   python3 simulado_n1_desafios.py
   ```

Critérios de Avaliação da AVP1 (Aula 10)

- **Integração Poliglota:** Lógica de leitura/escrita coordenada entre Redis e MongoDB.
- **Cache-Aside com TTL:** Consulta primária ao cache e hidratação com expiração temporal.
- **Operadores Atômicos:** Modificação de concorrência com `$inc` e **`HINCRBY`**.
- **Indexação ESR:** Justificativa técnica da ordem dos campos no índice composto.

Plantão de Dúvidas & Simulado N1

IFCE Campus Tauá – Preparatório para AVP1 e AVT1

Sucesso nas avaliações do primeiro período!